

# Лабораторная работа №4:

## Стохастическая оптимизация в машинном обучении.

### 1 Введение

В предыдущих лабораторных работах мы использовали полные (детерминированные) градиенты. Для задач машинного обучения целевая функция (эмпирический риск) имеет вид суммы по всем  $m$  объектам обучающей выборки:

$$F(x) = \frac{1}{m} \sum_{i=1}^m f_i(x) + \frac{\lambda}{2} \|x\|_2^2 \quad (1)$$

Вычисление полного градиента  $\nabla F(x)$  требует  $\mathcal{O}(m \cdot n)$  операций. В современных задачах размер выборки  $m$  может достигать десятков миллионов. В этом случае делать даже один шаг полного градиентного спуска или метода Ньютона становится вычислительно невозможно.

Идея стохастической оптимизации заключается в том, чтобы на каждой итерации использовать не истинный градиент, а его случайную несмещенную оценку, вычисленную по небольшому подмножеству данных (мини-батчу).

### 2 Алгоритмы

#### 2.1 Mini-batch SGD и проблема дисперсии

Вместо вычисления градиента по всей выборке, метод Стохастического Градиентного Спуска (SGD) на каждой итерации  $k$  равновероятно выбирает подмножество индексов  $\mathcal{I}_k \subset \{1, \dots, m\}$  размера  $b = |\mathcal{I}_k|$  и делает шаг:

$$x_{k+1} = x_k - \alpha_k g_k, \quad \text{где } g_k = \frac{1}{b} \sum_{i \in \mathcal{I}_k} (\nabla f_i(x_k) + \lambda x_k) \quad (2)$$

Так как выборка случайна, математическое ожидание стохастического градиента совпадает с полным градиентом:  $\mathbb{E}[g_k] = \nabla F(x_k)$ . Однако дисперсия этой оценки не равна нулю:  $\mathbb{E}[\|g_k - \nabla F(x_k)\|^2] = \sigma^2 > 0$ .

**Фундаментальная проблема SGD:** Из-за наличия шума  $\sigma^2$  стохастический градиент не стремится к нулю в точке оптимума  $x^*$ . Если использовать постоянный шаг  $\alpha$ , SGD не сойдется к оптимуму, а будет вечно блуждать в его окрестности, радиус которой пропорционален  $\alpha \sigma^2$ . Для достижения сходимости необходимо, чтобы длина шага убывала ( $\sum \alpha_k = \infty, \sum \alpha_k^2 < \infty$ ). Но убывающий шаг катастрофически замедляет метод (сублинейная сходимость  $\mathcal{O}(1/k)$  даже для сильно выпуклых задач).

#### 2.2 Методы сокращения дисперсии (SVRG)

Можно ли получить экспоненциальную (линейную) скорость сходимости  $\mathcal{O}(e^{-ck})$ , вычисляя только стохастические градиенты? Да, если мы сможем динамически уменьшать дисперсию оценки  $\sigma^2 \rightarrow 0$  по мере приближения к оптимуму.

Эту идею реализует метод **SVRG (Stochastic Variance Reduced Gradient)**. Метод работает эпохами (внешними циклами). В начале эпохи фиксируется точка  $\tilde{x}$  (якорь) и

вычисляется полный точный градиент  $\nabla F(\tilde{x})$ . Внутри эпохи делаются быстрые стохастические шаги (например,  $m$  шагов), но градиент корректируется с помощью якоря:

$$g_k = \nabla f_{i_k}(x_k) - \nabla f_{i_k}(\tilde{x}) + \nabla F(\tilde{x}) \quad (3)$$

Покажем, что эта оценка несмещенная:  $\mathbb{E}[g_k] = \nabla F(x_k) - \nabla F(\tilde{x}) + \nabla F(\tilde{x}) = \nabla F(x_k)$ .

Но её главное свойство в другом: по мере того, как алгоритм сходится ( $x_k \rightarrow x^*$ , и следовательно  $\tilde{x} \rightarrow x^*$ ), разность  $\nabla f_{i_k}(x_k) - \nabla f_{i_k}(\tilde{x})$  стремится к нулю, т.е. стохастический шум исчезает сам собой. Это позволяет SVRG сходиться с постоянным шагом  $\alpha$ , обеспечивая феноменальную скорость работы.

## 2.3 Адаптивные методы (Adam)

Обычный SGD обновляет все координаты вектора  $x$  с одинаковым шагом  $\alpha$ . Для разреженных данных (где одни признаки встречаются часто, а другие – очень редко) или плохо обусловленных задач это крайне неэффективно. Адаптивные методы индивидуально масштабируют шаг для каждой координаты обратно пропорционально исторической дисперсии градиентов по этой координате.

Наиболее продвинутым из адаптивных методов является **Adam** (Adaptive Moment Estimation). Он поддерживает экспоненциально сглаженные оценки первого момента  $m_k$  (инерция) и второго нецентрального момента  $v_k$  (квадрат градиента):

$$m_k = \beta_1 m_{k-1} + (1 - \beta_1) g_k \quad (4)$$

$$v_k = \beta_2 v_{k-1} + (1 - \beta_2) g_k \odot g_k \quad (\text{покомпонентный квадрат}) \quad (5)$$

$$\hat{m}_k = \frac{m_k}{1 - \beta_1^k}, \quad \hat{v}_k = \frac{v_k}{1 - \beta_2^k} \quad (\text{коррекция смещения нуля}) \quad (6)$$

$$x_{k+1} = x_k - \frac{\alpha}{\sqrt{\hat{v}_k} + \epsilon} \odot \hat{m}_k \quad (\text{покомпонентное деление}) \quad (7)$$

Деление на  $\sqrt{\hat{v}_k}$  работает как автоматический диагональный предобуславливатель, выравнивая линии уровня целевой функции (аналог диагонали матрицы Гессе).

## 3 Формулировка задания

Как и в предыдущих лабораторных работах, задание выполняется в командах по 3 человека. Каждая команда проводит базовые эксперименты на гладких ML-оракулах из своего варианта (закрепленного с Лабораторной №1) с  $L_2$ -регуляризацией и выполняет индивидуальный исследовательский трек по вариантам из Приложения А. Реализуемые методы должны поддерживать работу с мини-батчами. В отчете необходимо явно указать, как были распределены задачи между участниками команды.

### 3.1 План выполнения

1. Реализуйте базовый метод стохастического градиентного спуска. На каждой итерации выбирайте  $b$  случайных индексов без возвращения. Вам необходимо реализовать три стратегии расписания длины шага:

(a) **Constant:** Постоянный шаг  $\alpha_k = \alpha_0$ .

(b) **Inverse Sqrt:** Убывающий шаг  $\alpha_k = \frac{\alpha_0}{\sqrt{k+1}}$ .

(с) **Step Decay:** Экспоненциальное затухание  $\alpha_k = \alpha_0 \cdot \gamma^{\lfloor \text{epoch} / \text{drop\_freq} \rfloor}$  (например, уменьшение шага в 2 раза каждые 10 эпох).

2. Реализуйте метод SVRG. Алгоритм имеет двухцикловую структуру:

- **Внешний цикл (смена эпохи):** Сохраняет текущую точку как «якорь»  $\tilde{x} = x_k$  и вычисляет полный точный градиент  $\nabla F(\tilde{x})$  (стоимость: 1 эпоха).
- **Внутренний цикл:** Делает  $N_s$  стохастических шагов (или  $N_s/b$  шагов мини-батчами) с использованием скорректированного градиента (3) по мини-батчу и постоянного шага  $\alpha$ . Обычно длина внутреннего цикла  $N_s$  выбирается равной  $m$  или  $2m$ .
- Новая точка якоря выбирается, как последняя точка внутреннего цикла  $x_{N_s}$  (или как среднее арифметическое по пройденной внутренней траектории).

3. Реализуйте метод Adam. Убедитесь, что все операции деления, умножения и извлечения корня выполняются покомпонентно (в numpy это обычные операторы `*`, `/`, `**0.5`). Не забудьте реализовать механизм коррекции смещения  $\hat{m}_k, \hat{v}_k$ , который критически важен на первых итерациях (иначе алгоритм сделает огромный шаг в начале). Параметры по умолчанию:  $\beta_1 = 0.9, \beta_2 = 0.999, \epsilon = 10^{-8}$ .
4. Проведите эксперименты, описанные в разделах 3.2 – 3.4. Оформите результаты в отчет.
5. Выполните один индивидуальный исследовательский трек на всю команду (см. Приложение А).

**Эффективные эпохи:** Поскольку методы используют разный объем данных на итерацию (SVRG считает полный градиент, SGD берет батч), ось X на всех графиках должна отображать число эффективных проходов по данным (эпох).

1 эпоха =  $m$  вычислений градиента  $\nabla f_i(x)$ . (Например, вычисление полного градиента — это 1 эпоха. Шаг SGD с батчем размера  $b$  - это  $b/m$  эпохи).

## 3.2 Эксперимент 1

Цель: Увидеть "шумовую окрестность" и понять, почему шаг должен убывать.

1. Возьмите ваши ML-оракулы и на каждом датасете обучите SGD (батч  $b \approx 10..50$ ) с тремя разными стратегиями шага (Constant, Inverse Sqrt, Step Decay). Подберите базовый шаг  $\alpha_0$  для каждой стратегии.
2. Для референса запустите полный градиентный спуск.
3. Постройте график зависимости значения функции  $F(x_k)$  от числа эффективных эпох.

**Вопросы:** Наблюдаете ли вы осцилляции у SGD с постоянным шагом в конце работы? Какое расписание длины шага позволило опуститься глубже всего по функции потерь? Почему на самых первых эпохах стохастический градиент падает по функции значительно быстрее, чем полный GD?

### 3.3 Эксперимент 2

Цель: Доказать на практике, что сокращение дисперсии восстанавливает линейную скорость сходимости.

1. Возьмите хорошо обусловленную задачу (добавьте значимый  $L_2$ -регуляризатор  $\lambda$ ).
2. Найдите точный оптимум  $F^*$  (запустив полный GD или L-BFGS с высокой точностью).
3. Запустите SGD с оптимальным убывающим шагом, обычный GD и SVRG (используйте постоянный шаг, длину внутреннего цикла  $N_s = m$ ).
4. Постройте график логарифма невязки  $\log(F(x_k) - F^*)$  от числа эффективных эпох.

**Вопросы:** Продemonстрировал ли SVRG линейную сходимость в отличие от SGD? Окупается ли высокая стоимость вычисления полного градиента в начале каждой эпохи SVRG итоговой скоростью?

### 3.4 Эксперимент 3

Цель: Показать, как адаптивные методы автоматически масштабируют пространство признаков, а также продемонстрировать их математический изъян на специальном контрпримере

1. Создайте искусственно «испорченный» датасет: возьмите ваши данные и домножьте первую половину признаков (столбцов матрицы  $A$ ) на 1000, а вторую половину разделите на 1000. Это сделает линии уровня целевой функции вытянутыми эллипсами.
2. Запустите на этих испорченных данных базовый SGD (с тщательной настройкой шага) и Adam (со стандартным шагом  $\alpha = 0.001$ ).
3. Постройте графики сходимости  $F(x_k)$  от эпох.

Согласно теории<sup>1</sup>, из-за экспоненциального сглаживания ( $\beta_2 < 1$ ) Adam "забывает" старые большие градиенты. Если алгоритм получает частые мелкие градиенты в одну сторону и редкие большие градиенты в другую, знаменатель  $\sqrt{v_k}$  становится слишком маленьким на редких градиентах. Это делает шаг по ним аномально огромным, что приводит к расходимости алгоритма.

1. Возьмите ваш оригинальный датасет. Выберите всего 1% случайных объектов.
2. Превратите их в «тяжелые аномалии»: инвертируйте их целевую переменную (замените класс 1 на  $-1$  или умножьте  $y$  на  $-1$  для регрессии) и умножьте значения их признаков на очень большое число (например,  $C = 100$ ). Остальные 99% данных оставьте без изменений.
3. Запустите Adam (используйте малый размер батча  $b = 5$ ,  $\beta_2 = 0.99$ ) на большое число эпох. Вы увидите, что функция потерь не стабилизируется, а периодически испытывает скачки, когда в батч попадает аномалия.

---

<sup>1</sup><https://openreview.net/pdf?id=ryQu7f-RZ>

**Вопросы:** Почему SGD расходится или застревает на таких данных? Покажите математически, анализируя формулу обновления Adam, как знаменатель  $\hat{v}_k^{-1/2}$  в Adam выступает в роли автоматического диагонального предобуславливателя матрицы Гессе, компенсируя разный масштаб признаков. Почему обычный SGD без проблем проглатывает редкие аномалии (loss слегка дергается, но метод не расходится), в то время как Adam терпит огромные скачки?

### 3.5 Углубленное исследование: Индивидуальный трек команды

В данном разделе команда должна представить результаты исследования по своему индивидуальному треку (см. Приложение А). Это задание требует постановки отдельного вычислительного эксперимента и глубокого математического анализа.

**Отчет по треку должен содержать:**

1. **Математическую постановку:** Вывод формул, доказательства или теоретическое описание исследуемой проблемы.
2. **Описание эксперимента:** Какие параметры варьировались, на каких данных проводились тесты.
3. **Результаты:** Графики, таблицы, профили времени работы.
4. **Выводы:** Глубокая аналитика полученных результатов. Почему метод вел себя именно так? Подтверждают ли результаты геометрию оптимизации из лекций?

## 4 Оформление задания

Работа команды сдается в виде трех взаимосвязанных артефактов (см. Лабораторная работа 1):

- Репозиторий на Github/Gitlab
- Отчет в PDF
- Презентация

Каждый проведенный эксперимент следует оформить как отдельный раздел в PDF документе (название раздела - название соответствующего эксперимента). Для каждого эксперимента необходимо сначала написать его описание:

- а) постановка задачи
- б) какие функции оптимизируются, каким образом генерируются данные, Описание датасетов (размерность  $m \times n$ , плотность/разреженность матриц), используемого "железа" (CPU/RAM), какие методы и с какими параметрами используются.
- в) далее должны быть представлены результаты соответствующего эксперимента - графики, таблицы и т. д.
- г) наконец, после результатов эксперимента должны быть написаны Ваши выводы - какая зависимость наблюдается и почему. Как это бьется с теорией оптимизации из лекций?

**Важно:** Отчет не должен содержать никакого кода. Каждый график должен быть прокомментирован - что на нем изображено, какие выводы можно сделать из этого эксперимента. Обязательно должны быть подписаны оси. Если на графике нарисовано несколько кривых, то должна быть легенда. Сами линии следует рисовать достаточно толстыми, чтобы они были хорошо видимыми.

# Приложение А. Индивидуальные исследовательские треки

## Трек 1. SGD с Усреднением Поляка-Рупперта

### Описание

Из-за дисперсии градиента точка  $x_k$  в методе SGD с постоянным шагом навсегда остается в окрестности оптимума, непрерывно осциллируя. Чтобы метод сошелся точно в оптимум, необходимо уменьшать шаг ( $\alpha_k \rightarrow 0$ ), что катастрофически замедляет скорость сходимости.

В 1992 году Б.Т. Поляк и Д. Рупперт независимо доказали теорему<sup>2</sup>: если в качестве финального ответа выдавать не последнюю точку  $x_k$ , а среднее арифметическое всей пройденной траектории  $\bar{x}_k = \frac{1}{k} \sum_{i=1}^k x_i$ , то метод сходится асимптотически с той же скоростью, как если бы мы знали точную матрицу Гессе. Усреднение работает как фильтр низких частот, математически нивелируя дисперсию стохастического градиента.

### Задание

1. Выведите рекуррентную формулу бегущего среднего:  $\bar{x}_k = \frac{k-1}{k} \bar{x}_{k-1} + \frac{1}{k} x_k$ . Это позволяет поддерживать усредненную точку на лету за  $\mathcal{O}(n)$  операций, не храня в памяти всю историю векторов.
2. Встройте поддержку усреднения в ваш базовый SGD. Добавьте гиперпараметр `burn_in` (в эпохах) – число шагов, которые алгоритм проходит до включения усреднения (период «разогрева»).
3. Запустите три алгоритма: SGD с постоянным шагом, SGD с убывающим шагом ( $\sim 1/\sqrt{k}$ ) и SGD с усреднением Поляка-Рупперта на ваших данных.

### Вопросы для анализа в отчете

1. Постройте график логарифма невязки по функции  $F(\bar{x}_k)$ . Покажите визуально, как усреднение траектории гасит шумовую окрестность при постоянном шаге.
2. Продемонстрируйте экспериментально важность параметра `burn_in`. Что происходит с качеством усредненной точки, если начать усреднение с самой первой итерации  $k = 1$  (когда  $x_0$  еще очень далеко от оптимума)?

## Трек 2. SAGA: Сокращение дисперсии без внешних циклов

### Описание

Метод SVRG требует полного вычисления градиента раз в эпоху, что приводит к ступенчатому графику сходимости и "простоям". Метод SAGA<sup>3</sup> обходит эту проблему за счет использования дополнительной памяти. Он хранит в памяти последний вычисленный вектор градиента для каждого из  $m$  объектов выборки (матрица размера  $m \times n$ ). На каждой итерации

---

<sup>2</sup>[https://www.researchgate.net/publication/236736831\\_Acceleration\\_of\\_Stochastic\\_Approximation\\_by\\_Averaging](https://www.researchgate.net/publication/236736831_Acceleration_of_Stochastic_Approximation_by_Averaging)

<sup>3</sup><https://arxiv.org/pdf/1407.0202>

1. Выбирается случайный индекс  $i$ .
2. Вычисляется новый градиент  $\nabla f_i(x_k)$ .
3. Формируется несмещенная оценка:  $g_k = \nabla f_i(x_k) - G[i] + \frac{1}{m} \sum_{j=1}^m G[j]$ .
4. Таблица обновляется:  $G[i] \leftarrow \nabla f_i(x_k)$ .

### Задание

1. Реализуйте алгоритм SAGA. (Внимание: хранение матрицы  $m \times n$  потребует оперативной памяти, поэтому используйте несамый большой из ваших датасетов).
2. Напишите эффективный рекуррентный пересчет среднего значения таблицы (без суммирования всех  $m$  строк на каждой итерации).
3. Сравните SAGA и SVRG по времени сходимости в секундах и числу эффективных эпох.

### Вопросы для анализа в отчете

1. Докажите математически, что формула градиента SAGA дает несмещенную оценку полного градиента  $\mathbb{E}[g_k] = \nabla F(x_k)$ .
2. Постройте графики сходимости. Как SAGA сглаживает ступенчатость, присущую SVRG? Кто из них сходится быстрее в секундах с учетом накладных расходов на работу с памятью?

## Трек 3. Выборка по важности

### Описание

В классическом SGD объекты мини-батча выбираются с равной вероятностью  $p_i = 1/m$ . Но объекты выборки неравноценны: у одних предсказание уже идеальное (градиент близок к нулю), у других - градиент меняется резко (большая локальная константа Липшица  $L_i$ ). Идея выборки по важности<sup>4</sup> заключается в том, чтобы сэмплировать объекты с вероятностью, пропорциональной их  $L_i$ . Теоретически доказано, что это строго минимизирует дисперсию стохастического градиента.

### Задание

1. Для ваших ML-функций оцените константы Липшица градиента  $L_i$  для каждого из  $m$  объектов обучающей выборки (через вычисление максимального собственного значения гессиана, для линейных моделей скалярное произведение признаков  $z_i = \langle a_i, x \rangle$  встроено в них линейно. Поэтому гессиан любого одного объекта всегда имеет вид матрицы ранга 1).
2. Сформируйте распределение вероятностей  $p_i = \frac{L_i}{\sum L_j}$ .
3. Реализуйте SGD, в котором индексы выбираются случайным образом согласно вероятностям  $p_i$ . **Важно:** чтобы оценка оставалась несмещенной, градиент выбранного объекта нужно разделить на  $(m \cdot p_i)$ :  $g_k = \frac{1}{m \cdot p_{i_k}} \nabla f_{i_k}(x_k)$ .

<sup>4</sup><http://proceedings.mlr.press/v37/zhaoa15.pdf>

## Вопросы для анализа в отчете

1. Докажите математически, что скорректированная оценка градиента остается несмещенной  $\mathbb{E}_{i \sim p}[g_k] = \nabla F(x_k)$ .
2. Сравните на графиках равномерный SGD и SGD с выборкой по важности. Удалось ли снизить шумовую окрестность в конце алгоритма?
3. На каких данных этот подход дает наибольший выигрыш: на идеально однородных или на данных с сильными выбросами в значениях признаков? Почему?

## Трек 4. Стохастический линейный поиск

### Описание

В детерминированной оптимизации мы не подбираем шаг  $\alpha$  вручную, полагаясь на условие Армихо (линейный поиск). Для SGD классический линейный поиск невозможен, так как вычисление полной функции  $F(x)$  на каждом шаге уничтожит всю суть стохастичности. Однако был предложен Стохастический линейный поиск (SLS)<sup>5</sup>, который проверяет условие Армихо строго на текущем мини-батче  $\mathcal{I}$ :

$$f_{\mathcal{I}}(x_k - \alpha \nabla f_{\mathcal{I}}(x_k)) \leq f_{\mathcal{I}}(x_k) - c \cdot \alpha \|\nabla f_{\mathcal{I}}(x_k)\|^2 \quad (8)$$

Это делает длину шага адаптивной к локальной геометрии без ручной настройки.

### Задание

1. Реализуйте процедуру бэктрекинга для стохастического градиента. Если условие не выполнено, шаг уменьшается  $\alpha \leftarrow \gamma \alpha$ . (Важно: выборка батча  $\mathcal{I}$  фиксирована на время цикла линейного поиска).
2. Встройте SLS в обычный SGD. Для инициализации шага на новой итерации используйте шаг с предыдущей итерации, умноженный на константу  $\rho > 1$  (например, 1.1), чтобы метод мог «разгоняться».
3. Сравните SLS с лучшим ручным расписанием шага для SGD.

## Вопросы для анализа в отчете

1. Постройте график изменения длины шага  $\alpha_k$  от номера эпохи. Автоматически ли SLS "понимает" что шаг нужно уменьшать при приближении к шумовой окрестности оптимума?
2. Почему для работы SLS размер батча  $b$  не должен быть слишком маленьким? Как шум батча влияет на корректность условия Армихо?

---

<sup>5</sup><http://papers.neurips.cc/paper/8630-painless-stochastic-gradient-interpolation-line-search-and-c.pdf>



## Трек 5. Ускоренный SVRG (Katyusha)

### Описание

Классический SVRG дает линейную скорость сходимости, но эта скорость не является ускоренной (в смысле Нестерова). В 2017 году Зейуан Аллен-Чжу предложил метод **Katyusha**<sup>6</sup>, который объединяет сокращение дисперсии (SVRG) и momentum Нестерова.

Katyusha поддерживает три последовательности:  $y_k$  (точка градиента),  $z_k$  (инерция) и  $x_k$  (основная точка). Главная инновация - так называемый "Katyusha momentum" который притягивает точку  $x_{k+1}$  не только к инерции, но и к якорю  $\tilde{x}$  из внешнего цикла SVRG. Это не дает дисперсии взорваться из-за экстраполяции Нестерова.

### Задание

1. Изучите статью и реализуйте алгоритм Katyusha Katyusha<sup>ns</sup> (без проксимального оператора, для гладких функций).
2. Шаги внутри внутреннего цикла выглядят как взвешенные выпуклые комбинации  $y_k$ ,  $z_k$  и якоря  $\tilde{x}$ .
3. Запустите базовый SVRG, Ускоренный градиентный спуск Нестерова (если вы его реализовывали в треке по выбору Лабораторной работы №2) и Katyusha на ваших задачах.

### Вопросы для анализа в отчете

1. Приведите в отчете полные формулы пересчета точек в методе Katyusha. За что отвечает параметр  $\tau_1$  (Katyusha momentum) в формуле выпуклой комбинации?
2. Постройте графики сходимости. Докажите экспериментально, что алгоритм Katyusha наследует лучшие черты обоих родителей: он сходится быстрее SVRG на плохо обусловленных задачах (благодаря инерции) и не осциллирует (благодаря сокращению дисперсии).

## Трек 6. Subsampled Newton (SSN)

### Описание

Вычислить полный гессиан  $\nabla^2 F(x)$  и обратить его за  $\mathcal{O}(n^3)$  практически нереализуемая в ML задача. Но мы можем вычислить гессиан по небольшой случайной подвыборке  $\mathcal{S}$  (subsample), получив стохастическую оценку гессиана  $H_{\mathcal{S}} = \frac{1}{|\mathcal{S}|} \sum_{i \in \mathcal{S}} \nabla^2 f_i(x_k)$ . Использование  $H_{\mathcal{S}}^{-1}$  вместо единичной матрицы дает метод стохастического второго порядка Subsampled Newton<sup>7</sup>. Интересно, что размер батча для гессиана  $|\mathcal{S}|$  может быть значительно меньше размера батча для градиента.

<sup>6</sup><https://www.jmlr.org/papers/volume18/16-410/16-410.pdf>

<sup>7</sup>[https://www.stat.berkeley.edu/~mmahoney/pubs/ssn\\_MathProg.pdf](https://www.stat.berkeley.edu/~mmahoney/pubs/ssn_MathProg.pdf)

## Задание

1. Реализуйте метод SSN. На каждом шаге выбирайте батч  $\mathcal{I}$  для градиента и отдельный независимый батч  $\mathcal{S}$  для гессиана.
2. Шаг метода:  $x_{k+1} = x_k - \alpha(H_{\mathcal{S}} + \gamma I)^{-1} g_{\mathcal{I}}$ . Добавление регуляризации  $\gamma I$  (где  $\gamma$  - малое число) обязательно, чтобы случайная матрица не оказалась вырожденной.
3. Сравните SSN и батчевый SGD на задаче с небольшой размерностью (чтобы обращение матрицы было быстрым).

## Вопросы для анализа в отчете

1. Сравните методы на графиках: по числу эффективных эпох и по реальному времени работы. Экономит ли стохастический второй порядок реальное время?
2. зафиксируйте батч градиента  $\mathcal{I}$ , и меняйте размер батча гессиана  $\mathcal{S}$  (например, 1%, 5%, 20% от датасета). Насколько точно нужно оценивать гессиан, чтобы метод сохранял правильное ньютоновское направление?

## Трек 7. Random Reshuffling

### Описание

В классической теории стохастической оптимизации доказывается сходимость метода SGD при условии, что объекты на каждой итерации выбираются независимо и равновероятно с возвращением. Однако на практике во всех фреймворках машинного обучения (PyTorch, TensorFlow) класс `DataLoader` с параметром `shuffle=True` работает иначе. В начале каждой эпохи датасет случайным образом перемешивается, и затем алгоритм проходит по нему строго последовательно без возвращения.

То есть теоретики анализировали SGD с возвращением, а практики использовали без возвращения. Лишь недавно <sup>8</sup> было строго доказано, что Random Reshuffling математически быстрее обычного SGD. Секрет кроется во внутреннем эффекте «сокращения дисперсии»: при проходе по датасету без возвращения сумма всех стохастических градиентов за эпоху строго равна полному градиенту эпохи. Алгоритм гарантированно "ощупывает" каждую точку данных ровно один раз, что не дает ему топтаться на месте, в то время как классический SGD может случайно пропустить некоторые объекты и взять другие по несколько раз, увеличивая дисперсию.

### Задание

1. Реализуйте две стратегии формирования мини-батчей для вашего базового SGD:
  - **Классический SGD:** на каждой итерации генерируются случайные индексы от 1 до  $m$  (выборка *с возвращением*, `np.random.choice(m, size=b, replace=True)`).
  - **Random Reshuffling (RR):** в начале эпохи генерируется случайная перестановка индексов (`np.random.permutation(m)`), и батчи последовательно нарезаются из этого перемешанного массива.

---

<sup>8</sup>[https://proceedings.neurips.cc/paper\\_files/paper/2020/file/c8cc6e90ccbff44c9cee23611711cdc4-Paper.pdf](https://proceedings.neurips.cc/paper_files/paper/2020/file/c8cc6e90ccbff44c9cee23611711cdc4-Paper.pdf)

2. Возьмите хорошо обусловленную задачу (например, с сильным  $L_2$ -регуляризатором  $\lambda$ ).
3. Запустите обе стратегии из одной начальной точки  $x_0$ , используя одинаковое убывающее расписание шага (например,  $\alpha_k = \frac{\alpha_0}{\sqrt{k+1}}$ ). Оптимизируйте модели на протяжении 50-100 эпох.

### Вопросы для анализа в отчете

1. Постройте график логарифма невязки целевой функции от числа эпох для SGD и RR. Убедитесь (и покажите на графике), что траектория Random Reshuffling сходится к оптимуму значительно глубже и быстрее.
2. Если увеличить масштаб графика траектории RR, можно заметить, что внутри одной эпохи значение целевой функции может локально осциллировать (возрастать), но к концу эпохи оно строго падает. Как этот эффект «внутриэпохальной» коррекции объясняется свойством суммы градиентов?
3. Математическая теория показывает, что скорость сходимости обычного SGD в окрестности оптимума составляет  $\mathcal{O}(\frac{1}{k})$ , тогда как у Random Reshuffling она составляет  $\mathcal{O}(\frac{1}{k^2})$ . Опираясь на дисперсию оценки градиента, объясните, почему выборка без возвращения позволяет алгоритму сходиться асимптотически быстрее?